

CORTEX_MED

AI-Assisted Hospital Queue & Patient Triage Management System

A full-stack, real-time platform that uses AI to estimate patient urgency at booking time and orders every consultation queue by clinical priority — never by diagnosis, always with a human in control.

HealthTech

AI for Good

Healthcare Operations

Real-Time Systems

Full-Stack Web App

Team LeftOvers

Khandaker Ali Ariyan · Fairuz Lubana Binte Newaz Bohota

Version 2.0 · Groq Cloud Build

Prepared July 11, 2026

Table of Contents

1	Executive Summary	3
2	Project Overview	4
3	Problem Statement	5
4	Proposed Solution	6
5	Main Objectives	7
6	User Roles & Responsibilities	8
7	AI Module — Cortex AI Triage Engine (CATE)	9
8	AI Infrastructure (Groq Cloud — Latest)	10
9	System Architecture	11
10	Technology Stack	12
11	Component Deep Dive	13
12	Data Model Highlights	14
13	Real-Time Features	15
14	Security Features	16
15	Deployment & Hosting	17
16	Setup & Usage Instructions	18
17	Roadmap & Known Limitations	19
18	Future Improvements	20
19	Team & Project Vision	21

What changed in this build (v2.0)

Earlier versions ran a **self-hosted** LLM via Ollama / vLLM. The final hackathon build moves inference to the **Groq Cloud API running Meta Llama 3.1 8B Instant** for ~1-2 second responses, while keeping the exact same /triage contract and pipeline. Deployment is now cloud-native on **Vercel + Render + MongoDB Atlas**, with Docker Compose retained for local development.

1 Executive Summary

CORTEX_MED is a full-stack hospital operations platform that pairs a large language model with a real-time, priority-ordered consultation queue. The moment a patient is booked, their free-text symptoms are sent to a purpose-built AI triage microservice — the **Cortex AI Triage Engine (CATE)** — which returns a structured **urgency assessment, never a diagnosis**. That assessment drives queue ordering across every department, so receptionists and doctors always see the most clinically urgent patient next, while every connected dashboard and the patient's own public tracking page update instantly over Socket.IO.

The platform is built from four cooperating services — a Next.js patient & staff frontend, an Express + MongoDB backend, a FastAPI AI microservice, and a MongoDB Atlas datastore — and is designed to be **resilient by default**: if the AI is ever unreachable, booking and queueing continue normally on a neutral fallback priority. Final clinical judgment always remains with the medical professional.

The gap it closes

Walk-in patients are usually seen in arrival order, so a possible cardiac case can wait behind minor ailments. CORTEX_MED reorders the queue by AI-estimated urgency.

How it stays safe

The AI only outputs an urgency score with a full explainability trail. Low-confidence results are flagged for human review rather than acted on blindly.

2 Project Overview

CORTEX_MED is an AI-powered hospital management platform designed to improve how hospitals prioritize patients, manage consultation queues, and reduce waiting times. Instead of relying solely on a first-come, first-served system, the platform analyzes a patient's symptoms immediately after registration and estimates how urgently the patient should be treated.

The system **never diagnoses diseases or recommends treatments**. Instead, it provides doctors and receptionists with an AI-assisted urgency assessment that helps ensure patients with potentially life-threatening conditions are treated before non-critical cases. The platform also synchronizes patient queues, doctor dashboards, administrative analytics, and patient tracking pages in real time using Socket.IO.

At a glance

Dimension	Summary
Category	HealthTech · AI for Good · Healthcare Operations · Real-Time Systems · Full-Stack Web App
Core idea	AI-assisted triage that reorders consultation queues by clinical urgency, in real time
AI role	Decision-support only — urgency score + explainability, never diagnosis or treatment
Services	Frontend (Next.js) · Backend (Express) · AI Service (FastAPI + Groq) · Database (MongoDB Atlas)
Real-time layer	Socket.IO push updates to all dashboards and public tracking pages
Repository	github.com/khandakeraliariyan/CORTEX_MED
Live demo	cortex-med.vercel.app

3 Problem Statement

In busy hospitals and clinics, walk-in patients are typically seen in strict arrival order, regardless of how urgent their condition actually is. A patient experiencing a possible cardiac event can end up waiting behind several patients with minor ailments simply because they registered later. Most hospitals still rely on manual queue management, which leads to a real patient-safety gap.

Problem	Consequence
Patients treated strictly in arrival order	Critical cases wait behind non-urgent ones
Receptionists aren't clinically trained to assess urgency	Urgency is guessed, not measured
Doctors lack a real-time view of patient priorities	No consolidated picture of who needs to be seen next
Patients have no visibility into waiting times	Anxiety, crowding, and repeated front-desk queries
Administrators can't monitor queue efficiency	No data to optimize staffing or flow

Net effect
Longer waiting times, inefficient resource allocation, and increased risk for patients who need immediate attention. AI-assisted triage — used correctly, as a decision-support signal rather than a diagnostic authority — can help close this gap.

4 Proposed Solution

CORTEX_MED introduces an AI-assisted triage system integrated directly into hospital operations. The moment an appointment is booked, the platform runs an automated sequence:

#	Step
1	Patient details + free-text symptoms are submitted via the booking form.
2	The backend generates a unique appointment code (CORTEX_MED-####) and a sequential per-doctor token number.
3	Symptom text is sent to CATE (POST /triage) with a 20-second backend timeout guard.
4	CATE returns priority, confidence, explainability factors, risk level, recommended department, and a plain-language summary.
5	The patient is inserted into the correct doctor's queue, sorted by priority then arrival order.
6	Estimated wait times are recalculated for every patient in that queue.
7	Socket.IO broadcasts patient:booked, queue:updated, and wait:updated to every connected client.
8	Patients track their appointment on a public, no-login page.

Scoped narrowly and safely

The AI never diagnoses a disease, prescribes medicine, or recommends treatment. It only outputs an urgency score with a full explainability trail. If it is ever unreachable or returns something unparseable, both the AI service and the Express backend fall back to a neutral default (priority 3, "AI unavailable") — so a patient can always be booked and seen, AI or no AI.

5 Main Objectives

Reduce waiting time

Recompute estimated waits continuously and surface them to staff and patients.

Prioritize emergency patients

Move clinically urgent cases to the front of the queue automatically.

Improve hospital workflow

Give each role a purpose-built dashboard for their part of the flow.

Assist doctors with explainable AI

Show risk, confidence, and the symptoms driving each decision.

Improve patient experience

Live, no-login tracking of queue position and estimated wait.

Real-time hospital monitoring

Push queue changes to every connected screen instantly.

Practical AI integration

Demonstrate safe, decision-support AI embedded in real operations.

Human-in-the-loop by design

Final clinical decisions always remain with medical professionals.

6 User Roles & Responsibilities

The platform defines four roles. Three are authenticated staff roles (Admin, Doctor, Receptionist); the fourth, Patient, requires **no login** at all.

Administrator

- Manage doctors and receptionists
- Monitor AI engine status
- View hospital analytics & performance
- Configure hospital settings
- Monitor queues across departments

Doctor

- View assigned queue
- Call next patient / complete consultations
- Review AI-generated urgency assessment
- View patient history
- Monitor consultation statistics

Receptionist

- Register patients & book appointments
- Enter symptoms for triage
- View doctor availability
- Monitor live queues
- Track patient waiting times

Patient no login

- Receive an appointment code
- Track queue status live
- View estimated waiting time
- Monitor consultation progress

Access rules

Self-registration is limited to **doctor** and **receptionist** roles. Admin accounts cannot be self-created — registering as admin is rejected at the service layer (403) and provisioned another way. When a user registers as a doctor, a matching Doctor profile is auto-created and login returns their doctorId.

7 AI Module — Cortex AI Triage Engine (CATE)

CATE is a standalone **FastAPI microservice**, deliberately decoupled from the main backend so it can be swapped, scaled, or replaced independently. It assists receptionists and doctors by prioritizing the consultation queue — it does not diagnose, prescribe, recommend treatment, or replace a doctor.

Internal pipeline



Mirrored in a modular folder structure: `api` → `services` → `models` → `schemas` → `utils`, with automatic retry (linear backoff) and neutral fallback on failure.

AI output fields

Field	Meaning
Priority	P1 Critical P2 Urgent P3 Moderate P4 Mild P5 Non-urgent
Confidence	A 0–1 score attached to every assessment (e.g. 0.97).
Risk level	One of Low / Medium / High / Critical; derived from priority if the model omits it.
Department	Single best-fit specialist dept. from a fixed list; unknown values fall back to General Medicine.
Factors (explainability)	Short phrases naming the symptoms that drove the priority, rendered as “AI Reasoning” bullets.
Summary	Condenses a long symptom description into one or two sentences a doctor can read in seconds.

Example request / response

```
POST /triage
{ "symptoms": "Severe chest pain, difficulty breathing, heavy sweating, pain spreading to left arm" }

→ 200 OK
{
  "priority": 1,
  "reason": "Symptoms suggest a possible cardiac emergency requiring immediate attention.",
  "confidence": 0.97,
  "factors": ["Severe chest pain", "Difficulty breathing", "Pain radiating to left arm"],
  "risk": "Critical",
  "department": "Cardiology",
  "summary": "Possible cardiac emergency. Immediate cardiology assessment recommended."
}
```

Safety guardrail

The AI must never diagnose diseases, prescribe medicine, recommend treatment, or replace a doctor's judgment. It only estimates urgency, always attaches a confidence score, and a low-confidence result is visibly flagged for human review rather than acted on blindly. Priority scale: **1** critical · **2** urgent · **3** moderate · **4** mild · **5** non-urgent.

8 AI Infrastructure — Groq Cloud (Latest)

Earlier versions of the project supported **Ollama** and **vLLM** for local and AMD ROCm GPU deployments. For the final hackathon deployment, the AI service now runs inference against the **Groq Cloud API with Meta Llama 3.1 8B Instant**, keeping the exact same /triage interface and pipeline. Because the backend is selected purely by environment variables, swapping providers requires **zero changes** to the Express backend or the frontend.

Current AI stack

Component	Value
AI framework	FastAPI (Python, Uvicorn)
AI provider	Groq Cloud
Model	Meta Llama 3.1 8B Instant
Communication	REST API (POST /triage)
Response format	Structured JSON (validated & clamped)
Average response	~1–2 seconds (network dependent)

Runtime workflow



Configuration (environment variables)

Variable	Example	Description
PORT	8000	Port the FastAPI service listens on
CORS_ORIGINS	*	Allowed CORS origins
LLM_BACKEND	groq	groq (current) · ollama / vllm (legacy, local)
GROQ_API_KEY	gsk_...	Groq Cloud API key (server-side only)
LLM_MODEL	llama-3.1-8b-instant	Model name/tag requested from Groq
LLM_TIMEOUT_SECONDS	8	Timeout for a single LLM call
LLM_TEMPERATURE	0.2	Kept low for consistent, parseable JSON
MAX_RETRIES	1	Retries before falling back to neutral response
RETRY_BACKOFF_SECONDS	0.5	Linear backoff between retries

Legacy local backends use LLM_BASE_URL (e.g. `http://localhost:11434`) with model tags such as `llama3.1:8b-instruct-q4_0` (Ollama) or `llama3.1:8b-instruct` (vLLM).

Failure handling

Neutral fallback guarantees availability

If the LLM is unreachable or returns output that can't be parsed into valid JSON, CATE retries (MAX_RETRIES, default 1) with linear backoff, then falls back to a neutral response rather than raising an error: **priority 3**, reason "AI unavailable. Default priority assigned.", **confidence 0**, risk "Medium", department "General Medicine". This guarantees appointment booking never blocks on the AI being down. GET /health returns the configured backend and model without triggering inference; the Express backend proxies this at GET /api/v1/triage/engine-status (admin-only).

Data note (accurate for the cloud build)

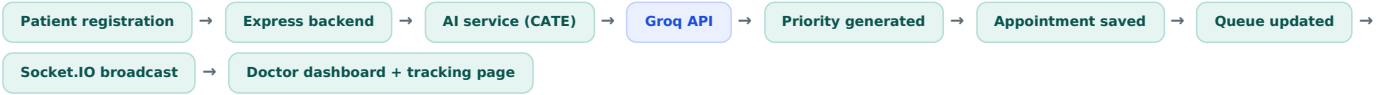
In the Groq build, only **free-text symptom descriptions** are sent to the Groq API to compute urgency — patient-identifying fields (name, phone, age) are **not** part of the triage payload. On the legacy self-hosted (Ollama/vLLM) build, inference stays entirely on local infrastructure.

9 System Architecture

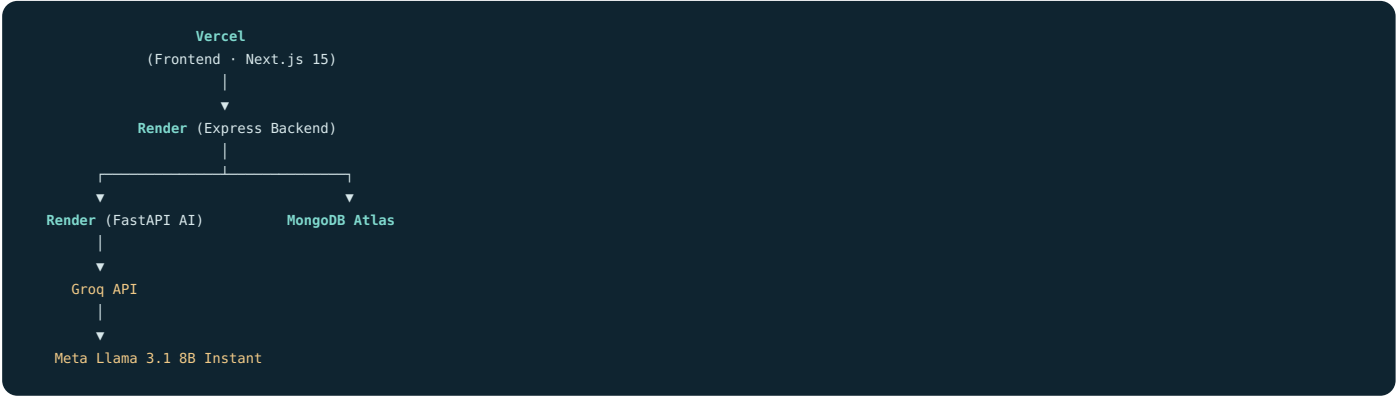
CORTEX_MED is a four-service system. The frontend talks to the backend over REST (/api/v1) and Socket.IO; the backend calls the AI service over a simple internal HTTP contract (POST /triage) that is identical regardless of which LLM backend is wired up.

Service	Role	Stack	Port
client	Staff dashboards + public tracking UI	Next.js 15, React 19, TypeScript, Tailwind v4, TanStack Query, Zustand, Socket.IO client	3000
server	REST API, auth, business logic, realtime hub	Node.js, Express 5, MongoDB/Mongoose 9, Socket.IO, JWT, Zod, bcrypt	5000
ai-service	AI triage engine (CATE)	Python, FastAPI, Uvicorn, Groq API (Llama 3.1) — Ollama/vLLM legacy	8000
mongo	Persistent data store	MongoDB Atlas (cloud) / MongoDB 7 (local)	27017

Request flow — booking an appointment



Deployment architecture (cloud)



10 Technology Stack

Frontend

Next.js 15 React 19 TypeScript Tailwind CSS v4
TanStack Query Zustand Socket.IO client

AI Service

Python FastAPI Uvicorn Groq API Meta Llama 3.1 8B
Ollama / vLLM (legacy)

Backend

Node.js Express 5 MongoDB Mongoose 9 Socket.IO Zod

Real-time & State

Socket.IO Zustand TanStack Query

Authentication & Security

JWT (access + refresh) bcrypt Role-based guards CORS

Deployment & Tooling

Vercel Render MongoDB Atlas Docker Docker Compose
Git / GitHub

Category & technology tags

Category: HealthTech · AI for Good · Healthcare Operations · Real-Time Systems · Full-Stack Web App

Technology: Next.js · React · TypeScript · Tailwind CSS · Node.js · Express · MongoDB · Mongoose · Socket.IO · JWT · Python · FastAPI · Llama 3.1 · Groq · Ollama · vLLM · Docker · Docker Compose · TanStack Query · Zustand

11 Component Deep Dive

11.1 Backend (server/)

A modular Express 5 application organized by domain: auth, doctor, appointment, triage, queue, wait-time, hospital, notification, plus scaffolded dashboard and analytics modules. Each implemented module follows a consistent five-file layout — route → controller → service → validation (Zod) → model — keeping HTTP handling, business logic, and schema validation cleanly separated.

AUTHENTICATION & AUTHORIZATION

- JWTs carry { id, email, role }, accepted as Authorization: Bearer <token> or a raw token.
- The auth(...roles) middleware throws 401 for missing/invalid/expired tokens and 403 when the role isn't allowed; auth() with no args only requires a valid token.
- Three roles: admin, doctor, receptionist. Access tokens are short-lived; POST /api/v1/auth/refresh exchanges a refresh token for a new access token.
- Registering as admin is rejected at the service layer (403) — admin accounts cannot be self-created.

KEY API SURFACE (BASE: /API/V1)

Module	Representative endpoints	Notes
auth	POST /register, /login, /refresh · PATCH /change-password · GET /me	Doctor registration auto-creates a Doctor profile; login returns doctorId.
doctors	GET / · POST / · PATCH /:id · DELETE /:id	Admin-managed; POST accepts an existing user ID or inline account details.
appointments	GET / · GET /track/:code (public) · POST /	Booking triggers AI triage + queue recalculation + socket broadcasts.
triage	POST / · GET /engine-status (admin)	engine-status proxies CATE /health without triggering inference.
queue	GET /:doctorId · PATCH /call-next/:doctorId · PATCH /complete/:appointmentId	Returns current patient, sorted waiting list, and live stats.
hospital-settings	GET / · PATCH / (admin)	Singleton document: hospitalName, facilityId.

Uniform response envelope

Every response — success or error — is shaped as { success, message, meta, data }. Thrown `AppError(statusCode, message)` instances are caught centrally by `globalErrorHandler` and returned in the same shape with `success: false`.

11.2 Frontend (client/)

A Next.js 15 App Router application in strict TypeScript, structured around two route groups — (public) and (protected) — with per-role guarded subtrees for admin, doctor, and receptionist. Route protection is currently client-side: each layout inspects auth state and redirects (no `middleware.ts` at the edge yet).

- **Provider composition:** ThemeProvider → QueryProvider → AuthProvider → SocketProvider. Order matters — SocketProvider reads the token the AuthProvider manages.
- **Server state:** TanStack React Query (staleTime 60s, gcTime 5m, refetch-on-focus off). Each feature exposes a `*.service.ts` and `use-*.ts` hooks.
- **Client state:** Zustand — `useAuthStore` (persists user + isAuthenticated) and `useNotificationsStore` (last 20 realtime events).
- **Polling safety net:** `useQueue` refetches every 10s and `useTrackAppointment` every 15s, independent of sockets — so screens stay correct even if an event is missed.
- **Auth flow:** a 401 triggers a transparent, deduped `/auth/refresh` and a single retry; failure clears tokens and redirects to `/login`.

11.3 AI Service (ai-service/)

A cleanly layered FastAPI project: `api/` (the single POST `/triage` route) → `services/` (orchestration pipeline) → `models/` (an HTTP client abstracting the Groq / OpenAI-compatible / Ollama request shapes) → `schemas/` (Pydantic request/response contracts) → `utils/` (JSON extraction, validation/clamping, async retry-with-backoff). This separation is exactly what lets the LLM backend be swapped via environment variables alone, with zero changes elsewhere.

12 Data Model Highlights

Collection	Key fields
User	name, email (unique), password (bcrypt, not selected by default), role (admin/doctor/receptionist), isActive, notificationPreferences { criticalAlerts, dailySummary, aiSuggestions }, lastLogin, timestamps
Doctor	user (ref, unique), department, specialty, room, consultationFee, avgConsultationTime (default 15), workingDays[], startTime/endTime, status (available/unavailable/on_leave), timestamps
Appointment	patientName, age, gender, phone, doctor (ref), symptoms, appointmentCode (unique), tokenNumber, priority (1-5), triageReason, triageConfidence (0-1), triageFactors[], riskLevel, recommendedDepartment, aiSummary, aiModel, triagedAt, estimatedWait, status (waiting/serving/completed/cancelled), calledAt, completedAt, notes, timestamps
HospitalSettings	Singleton: hospitalName (default "CortexMed Hospital"), facilityId, timestamps

13 Real-Time Features

Socket.IO pushes queue changes to every connected client, backing both live dashboard refreshes and an in-app notification feed.

Event	Payload	Emitted when
queue:updated	{ doctorId }	An appointment is created, called, or completed
wait:updated	{ doctorId }	Signals clients to refetch wait-time estimates
patient:booked	Full appointment object	POST /appointments
patient:called	Full appointment object	PATCH /queue/call-next/:doctorId
patient:completed	Full appointment object	PATCH /queue/complete/:appointmentId

Events are currently broadcast globally (io.emit) rather than room-scoped, and no auth handshake or room-joining is required to connect — a deliberate hackathon-stage simplification noted in the roadmap.

14 Security Features

JWT authentication

Short-lived access tokens + refresh tokens carrying id / email / role claims.

Password hashing

bcrypt; passwords never selected by default in queries.

Role-based authorization

admin / doctor / receptionist guards on routes and APIs.

Protected routes

Guarded route groups for authenticated experiences.

Input & API validation

Zod schemas validate every request payload.

CORS protection

Configurable allowed origins across services.

15 Deployment & Hosting

The final build is **cloud-native**. It is also fully containerized: every service ships a Dockerfile and a single `docker-compose.yml` brings the whole stack up locally.

Cloud deployment (primary)

Service	Platform	URL / notes
Frontend	Vercel	<code>cortex-med.vercel.app</code>
Backend API	Render	Express REST + Socket.IO hub
AI service	Render	FastAPI → Groq API
Database	MongoDB Atlas	Cloud-hosted, managed cluster
Repository	GitHub	<code>github.com/khandakeraliariyan/CORTEX_MED</code>

Local containerized stack (development)

All services ship their own Dockerfile, orchestrated via `docker/docker-compose.yml` (which also runs MongoDB as a container). The entire stack starts with one command:

```
docker compose -f docker/docker-compose.yml up --build
```

Service	URL after startup
Frontend	<code>http://localhost:3000</code>
Backend API	<code>http://localhost:5000/api/v1</code>
AI triage service	<code>http://localhost:8000 (health at /health)</code>
MongoDB	<code>localhost:27017</code>

Swapping the AI backend

Because `/triage` is identical across providers, switching between Groq (cloud) and Ollama/vLLM (local/ROCm) is done entirely through a `docker/.env` override — no Express or frontend code changes are needed.

16 Setup & Usage Instructions

#	Step
1	Install Docker Desktop (or Docker Engine + Compose v2 on Linux).
2	Ensure server/.env exists with JWT secrets and required variables (the README's variable table serves as the template — no .env.example ships yet).
3	Set the AI provider: for the cloud build, provide LLM_BACKEND=groq + GROQ_API_KEY. For local inference, install Ollama and run ollama pull llama3.1:8b-instruct-q4_0, or point at a vLLM server via a docker/.env override.
4	Run docker compose -f docker/docker-compose.yml up --build from the repo root to bring up MongoDB, the AI engine, the backend, and the frontend together.
5	Open http://localhost:3000 for staff dashboards, or /track/<code> for public patient tracking (the tracking form performs the lookup by code).

Resilient by design

If the AI backend is unavailable at any point, appointment booking and queueing continue to work normally — triage simply falls back to the safe neutral default priority, so the demo is resilient even without a live LLM connection.

17 Roadmap & Known Limitations

As a hackathon-stage build, several areas are intentionally scoped for future iteration and documented transparently.

PLATFORM-WIDE

- No automated test suite, seed script, Postman collection, or OpenAPI/Swagger spec yet.
- No `.env.example` files ship for server, client, or docker/.

BACKEND

- No helmet, rate-limiting, or compression middleware on the Express app.
- `globalErrorHandler` doesn't yet specially format Zod errors into field-level 400s.
- `QueueService.estimateWait` exists but isn't wired to a route.
- `mongoose.connect` has no retry/reconnect handling.
- Dashboard and analytics backend modules are empty scaffolding.
- Socket.IO events are broadcast globally, with no auth handshake or room-joining.

FRONTEND

- Route protection is entirely client-side (no `middleware.ts`) — a brief pre-redirect flash, not a real boundary.
- Most UI lives in two large files (`pages.tsx`, `ui.tsx`) awaiting decomposition.
- Logout is local-only; the refresh token isn't server-revoked.
- Password-reset and a logged-in patient dashboard are scaffolded but unimplemented.
- Analytics are computed client-side rather than from a dedicated backend endpoint.

AI SERVICE

- Single prompting strategy and a fixed department list; multi-language input and configurable taxonomies are next.
- No persistent audit trail of individual triage calls beyond what the backend stores on the appointment.

18 Future Improvements

- Voice symptom input
- Bengali language support
- Electronic Health Records (EHR) integration
- SMS notifications
- Multi-hospital deployment
- Predictive patient-flow analytics
- Doctor scheduling optimization
- AI-assisted medical report generation
- Mobile application
- Appointment forecasting
- Queue prediction from historical data
- Configurable department taxonomies

19 Team & Project Vision

Team LeftOvers

- Khandaker Ali Ariyan
- Fairuz Lubana Binte Newaz Bohota

Repository & Demo

Code: github.com/khandakeraliariyan/CORTEX_MED

Live: cortex-med.vercel.app

Project Vision

"Empowering hospitals with AI-assisted decision support to reduce waiting time, improve patient prioritization, and enable smarter, faster, and more efficient healthcare — while ensuring that final clinical decisions always remain in the hands of medical professionals."

CORTEX_MED · AI-Assisted Hospital Queue & Patient Triage Management System · Version 2.0 (Groq Cloud Build) · Prepared July 11, 2026